Haas ST-10 Dashboard — Design- und Implementierungsplan

Stand: 17.10.2026 · Basis: `ThMRudolf/2023_ProdcutionCellITAM/dashboard/IIoT_Haas/Backend` (Commit `1d01283`)

---

0. Ausgangslage und Annahmen

Die ST-10 der Zelle wird hier als **Classic Haas Control (CHC), vor 2016** behandelt. Sie liefert also kein MTConnect, sondern nur Machine Data Collection (MDC) über RS-232 mit Q-Codes. Bevor irgendetwas gebaut wird, muss das am Gerät geklärt werden (Phase 0). Steckt doch eine NGC drin, entfallen Adapter und Agent komplett, und die ST-10 hängt sich wie die TMP1 direkt an das bestehende Backend.

Was das Referenz-Repo tatsächlich enthält:

Datei	Inhalt	Wiederverwendbar?
`Backend/main.py`	Flask-REST-API, Hintergrund-Poller auf `/current` + `/sample`, SQLite (WAL), Proxy `/proxy/<path>`, Ingest `/api/sensor`	Ja, mit Umbau (siehe 3.3)
`Backend/cors_proxy.py`	Statischer Server + CORS-Proxy für `index.html`, `app.js`, `styles.css`, `i18n.js`	Nur noch als Dev-Fallback
Frontend-Dateien	**fehlen im Repo**	Neu zu bauen

Zwei Inkonsistenzen im Bestand fallen sofort auf. Der Kopf von `main.py` spricht von einem Streamlit-Frontend, `cors_proxy.py` dagegen von einem Vanilla-JS-Frontend mit i18n. Dieser Plan entscheidet sich für Letzteres (Begründung in 4.1). Und das Agent-Ziel in `run_commands.txt` (`148.205.38.221:8082`) ist der native NGC-Agent — vermutlich die TMP1, nicht die ST-10.

---

1. Was am bestehenden Backend für die ST-10 bricht

Der Code funktioniert für die NGC-Fräse. Für die ST-10 sind fünf Stellen nicht tragfähig:

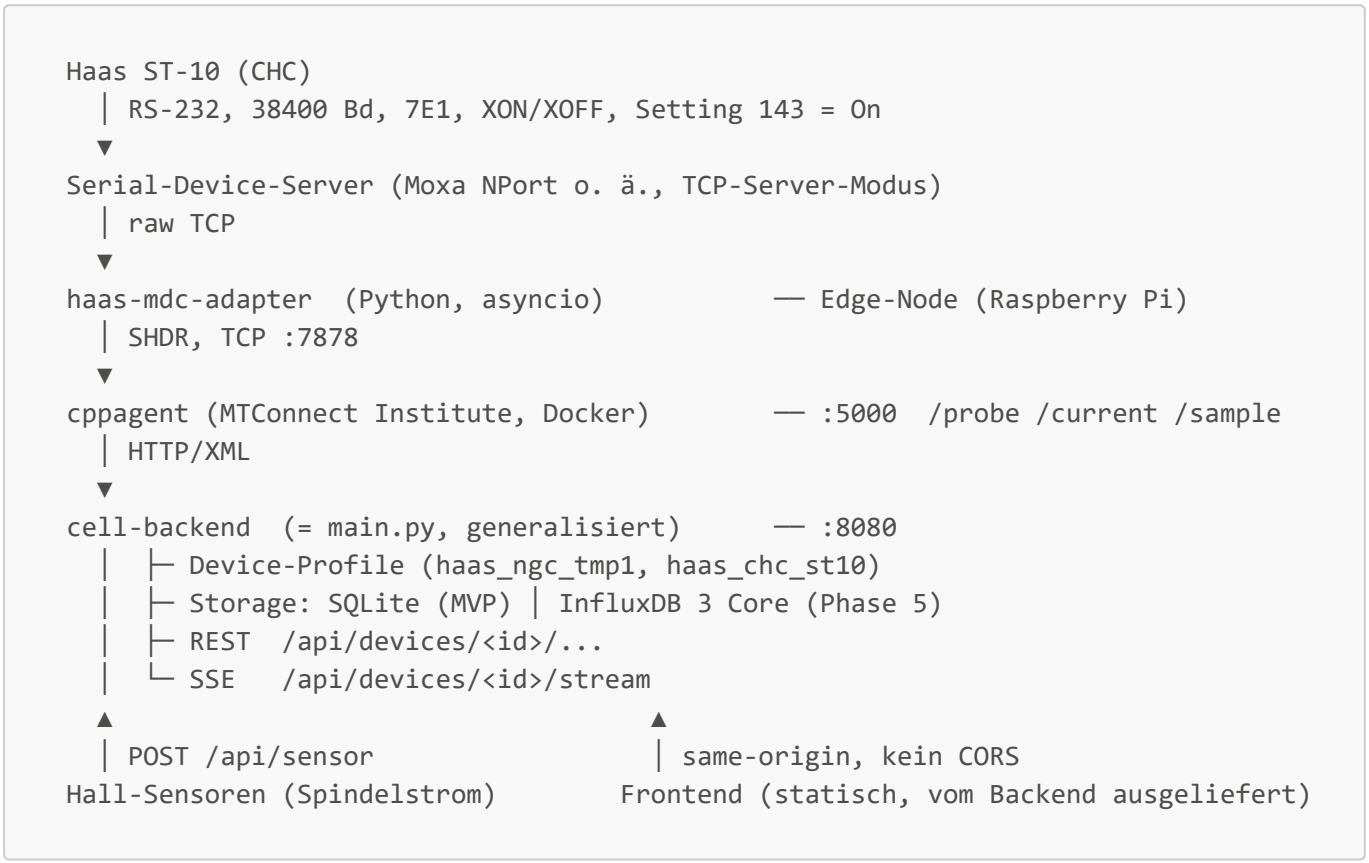
1. **Namespace fest auf MTConnect 1.2.** `NS = "urn:mtconnect.org:MTConnectStreams:1.2"`. Ein aktueller `cppagent` (2.x) liefert einen anderen Namespace, und `xml_all()` findet dann schlicht nichts — ohne Fehlermeldung. Der Namespace muss aus dem Root-Tag gelesen werden.
2. **Datenpunkt-Namen sind Haas-NGC-spezifisch.** `X_Axis_Actual_Position`, `DHMT_Codes`, `M30Counter1`, `SpindleMaxPower` existieren nur im Haas-eigenen Agent. Über einen Adapter heißen sie anders. Nötig ist eine Mapping-Schicht pro Gerät (Device-Profile, 3.2).
3. **Fräsen-Geometrie.** Die Trajektorie ist XYZ, die Statistik iteriert über X/Y/Z/C/A/B. Die ST-10 ist eine Drehmaschine: X (im Durchmesser programmiert), Z, Spindel, Revolverstation.
4. **`/sample` ohne `from`-Parameter.** Jeder Zyklus zieht den kompletten Puffer, verwendet aber nur das letzte Element. Geschwindigkeit wird als `|Δpos|/Δt` zwischen zwei Polls gerechnet. Bei der Q-Code-Rate der ST-10 (realistisch 1–4 Hz) ist diese Geschwindigkeit nur ein grober Indikator, keine Messgröße.

5. **Leistungsschätzung im Sensor-Ingest.** `power_est_w = 1.732 * 480 * I` ignoriert den Leistungsfaktor und setzt 480 V fest; `load_pct` nimmt 30 A Nennstrom an. Beides passt weder zum Netz in Mexiko-Stadt (220/440 V) noch zum Spindelmotor der ST-10. Richtig ist $P = \sqrt{3} \cdot U \cdot I \cdot \cos\phi$ mit konfigurierbarem `U`, $\cos\phi$ und `I_nenn`.

Kleinere Punkte: `if root_c and root_s` testet die Kinderanzahl eines `Element`, nicht `is not None`. Maschinen-Timestamp (Agent) und Backend-Timestamp (`datetime.now`) werden in derselben Zeile gemischt.

---

2. Zielarchitektur



Die zentrale Designentscheidung: **die ST-10 wird zu einem normalen MTConnect-Gerät gemacht**, statt dem Backend einen Sonderpfad für Q-Codes zu geben. Dadurch sprechen TMP1 und ST-10 dieselbe Sprache, das Frontend bleibt geräteunabhängig, und später können weitere Maschinen (oder Odoo, oder das Digital-Twin-Modell) denselben Agent lesen.

Gegenposition, kurz durchgespielt: Ein direkter Q-Code-Poller im Backend spart zwei Prozesse und einen Container. Das ist für ein einzelnes Studentenprojekt verlockend. Der Preis ist aber ein zweiter Datenpfad, den jede spätere Funktion (Speicherung, Twin-Kopplung, OPC-UA-Brücke) doppelt kennen muss. Für eine Learning Factory, die genau diese Standardisierung lehren soll, ist der Umweg über SHDR und `cppagent` der richtige.

2.1 Adapter: selbst bauen oder TrakHound?

`TrakHound/MTConnect.NET-Haas` (MIT) existiert, ist aber klein (6 Commits, keine Releases) und dokumentiert nur ein Ethernet-Modul auf Port 5051 — das ist der NGC-Weg. Für RS-232 an einer CHC ist der Funktionsumfang aus dem README nicht ersichtlich. Empfehlung: eigener Python-Adapter, ca. 300–400 Zeilen, voll im Repo und für Studierende lesbar. TrakHound bleibt Rückfalloption, falls Phase 0 zeigt, dass es die ST-10 sauber bedient.

2.2 Serieller Engpass und Abtastbudget

Bei 38400 Baud und 7E1 (10 Bit pro Zeichen) gehen rund 3 840 Zeichen/s über die Leitung. Eine `Q600`-Anfrage plus Antwort umfasst grob 35–40 Zeichen, also etwa 10 ms reine Übertragung. Wie lange die Steuerung intern für die Antwort braucht, weiß ich nicht; das wird in Phase 0 gemessen. Daraus folgt eine Abfrage in drei Gruppen:

Gruppe	Inhalt	Zielrate
schnell	X, Z, Spindeldrehzahl, Status aus `Q500`	2–4 Hz (nach Messung festlegen)
langsam	Werkzeug, Teilezähler, Zykluszeiten, Timer	0,2 Hz
statisch	Seriennummer, Modell, Softwarestand	einmal beim Verbindungsaufbau

Zwei praktische Fallen: Die MDC-Schnittstelle verträgt nur **einen** Client gleichzeitig (Berichte über „Too many connections" am Moxa). Und wird der RS-232-Port heute für DNC-Programmübertragung genutzt, kollidiert das mit MDC. Beides in Phase 0 klären.

Konsequenz für das Dashboard: Die Bahnkurve der ST-10 zeigt bei 2–4 Hz den groben Ablauf, keine Konturtreue. Das hochfrequente Prozesssignal ist der **Spindelstrom aus den Hall-Sensoren**; das Dashboard behandelt ihn entsprechend als Hauptsignal der Ansicht „Signale".

---

3. Backend und Datenpfad

3.1 Q-Code-Katalog (am Gerät zu verifizieren)

Die folgenden Codes sind aus der Haas-MDC-Dokumentation bekannt; welche davon die konkrete Softwareversion der ST-10 beantwortet, zeigt erst der Test in Phase 0.

Q-Code	Inhalt	→ MTConnect-Datenpunkt
`Q100`	Seriennummer	Device-Attribut
`Q101`	Softwareversion	Device-Attribut
`Q102`	Modell	Device-Attribut
`Q104`	Modus (MEM, MDI, …)	`ControllerMode`
`Q200`	Werkzeugwechsel gesamt	`ToolChangeCount` (Event, custom)
`Q201`	aktives Werkzeug	`ToolNumber`
`Q300`	Einschaltzeit	`AccumulatedTime` (POWERED)
`Q301`	Bewegungszeit	`AccumulatedTime` (CUT_TIME-Näherung)
`Q303`	letzte Zykluszeit	`ProcessTimer` (last)
`Q304`	vorherige Zykluszeit	`ProcessTimer` (previous)
`Q402` / `Q403`	M30-Zähler 1 / 2	`PartCount`

Q-Code	Inhalt	→ MTConnect-Datenpunkt
`Q500`	Programm, Status, Teilezahl in einer Antwort	`Program`, `Execution`, `PartCount`
`Q600 nnnn`	Makrovariable lesen	siehe unten

Makrovariablen für `Q600` (Nummern im Bedienhandbuch der ST-10 gegenprüfen): Maschinenposition X/Z aus dem `#5021`-Block, Werkstückposition aus dem `#5041`-Block, aktuelle Spindeldrehzahl `#3027`, Werkzeug in Arbeitsposition `#3026`. Für die Spindellast gibt es eventuell eine Makrovariable — welche, weiß ich nicht. Bis das geklärt ist, kommt Last ausschließlich aus dem Hall-Sensor.

Status-Mapping aus `Q500`: `BUSY` → `ACTIVE`, `IDLE` → `READY`, `ALARM ON` → `READY` + Condition `FAULT`, Timeout/keine Antwort → `UNAVAILABLE`. Alarmtexte liefert MDC nicht; das Dashboard zeigt deshalb nur „Alarm aktiv", nicht die Alarmnummer.

3.2 Adapter (`adapters/haas_mdc/`)

Aufbau:

```
haas_mdc/
  adapter.py        # asyncio-Hauptschleife, Gruppen-Scheduler
  transport.py      # SerialTransport (pyserial) | TcpTransport (Moxa)
  qcodes.py         # Request-Builder, Antwort-Parser (STX/ETB, ">", Kommas)
  shdr.py           # SHDR-Server :7878, Heartbeat (* PING / * PONG)
  config.yaml       # Port, Baudrate, Gruppen, Raten, Variablenliste
  replay.py         # spielt aufgezeichnete Q-Antworten ab (Simulator)
  tests/
```

Die SHDR-Ausgabe ist zeilenbasiert und nur bei Änderung:

```
2026-10-17T15:02:11.250Z|Xact|42.180|Zact|-12.405|S1speed|1850
2026-10-17T15:02:11.250Z|execution|ACTIVE|program|O01234|mode|AUTOMATIC
2026-10-17T15:02:15.000Z|tool|3|partcount|117
```

Zeitstempel setzt der Adapter beim Empfang der Antwort, nicht beim Absenden der Anfrage.

`replay.py` ist kein Beiwerk: Ohne Simulator kann niemand am Frontend arbeiten, solange die Maschine nicht verfügbar ist. In Phase 1 wird einmal eine 30-minütige Session an der echten ST-10 mitgeschnitten; daraus speist sich danach jede Entwicklung und jeder Test.

3.3 Agent (`agent/`)

`cppagent` als Container, `agent.cfg` mit Adapter `localhost:7878`, `Devices.xml` für die ST-10:

```
<Device id="st10" name="HaasST10" uuid="itam-haas-st10">
  <DataItems>
    <DataItem category="EVENT" id="avail" type="AVAILABILITY"/>
```

```
  </DataItems>
  <Components>
    <Axes id="ax">
      <Components>
        <Linear id="x" name="X">
          <DataItems>
            <DataItem category="SAMPLE" id="Xact" type="POSITION" subType="ACTUAL"
units="MILLIMETER"/>
          </DataItems>
        </Linear>
        <Linear id="z" name="Z"> … Zact … </Linear>
        <Rotary id="c" name="S1">
          <DataItems>
            <DataItem category="SAMPLE" id="S1speed" type="ROTARY_VELOCITY"
subType="ACTUAL" units="REVOLUTION/MINUTE"/>
          </DataItems>
        </Rotary>
      </Components>
    </Axes>
    <Controller id="ctrl"> … execution, mode, program, partcount, tool, system
condition … </Controller>
  </Components>
</Device>
```

Wichtig für die X-Achse: Haas-Drehmaschinen programmieren X im Durchmesser. Im Agent bleibt der Rohwert; die Umrechnung Durchmesser ↔ Radius gehört ins Device-Profile, damit sie an genau einer Stelle steht.

3.4 Generalisiertes Backend (`backend/`)

Aus `main.py` wird ein geräteunabhängiges Backend. Die Änderungen:

Device-Profile (`profiles/*.yaml`) übersetzen Agent-Datenpunkte in ein kanonisches Modell:

```yaml
# profiles/haas_chc_st10.yaml
device_id: st10
kind: lathe
agent: { host: 127.0.0.1, port: 5000, device: HaasST10 }
axes:
  X: { item: Xact, diameter_mode: true }
  Z: { item: Zact }
spindle:
  speed: S1speed
state:
  execution: execution
  mode: mode
  program: program
  part_count: partcount
  tool: tool
electrical:            # für Hall-Sensor-Ableitungen
  voltage_v: 220       # am Schaltschrank messen
```

```
    cos_phi: 0.85        # Typenschild Spindelmotor
    i_rated_a: null      # Typenschild — bis dahin keine Last-%-Anzeige
```

Die TMP1 bekommt `haas_ngc_tmp1.yaml` mit den heutigen NGC-Namen. Damit läuft der bestehende Pfad unverändert weiter.

Namespace dynamisch aus `root.tag` lesen. **`/sample?from=<nextSequence>`** statt Vollabzug, `nextSequence` aus dem Header merken. **Zeitstempel** durchgängig vom Agent übernehmen; Backend-Zeit nur als `ingested_at`.

Push statt Polling: Neues Endpoint `GET /api/devices/<id>/stream` als Server-Sent Events. Das Frontend fragt heute (impliziert durch `/api/latest`) jede Sekunde ab; SSE liefert Änderungen sofort und braucht keine zusätzliche Bibliothek. WebSockets wären symmetrisch, aber das Dashboard sendet nichts zurück außer gelegentlichen REST-Aufrufen — SSE reicht.

API-Vertrag (alte Routen bleiben als Alias auf das Default-Gerät erhalten, damit nichts Bestehendes bricht):

Methode	Route	Zweck
GET	`/api/devices`	Liste der Profile + Verbindungsstatus
GET	`/api/devices/<id>/latest`	kanonischer Snapshot
GET	`/api/devices/<id>/stream`	SSE: `snapshot`, `state`, `health`
GET	`/api/devices/<id>/series?signals=X,Z,rpm,i_spindle&from=…&to=…&max_points=2000`	Zeitreihen, serverseitig ausgedünnt
GET	`/api/devices/<id>/path?from=…&to=…`	XZ-Bahn
GET	`/api/devices/<id>/states?from=…&to=…`	Zustandsintervalle für die Zeitleiste
GET	`/api/devices/<id>/kpi?shift=current`	Verfügbarkeit, Teile/h, Zykluszeiten
GET	`/api/devices/<id>/health`	Adapter/Agent: Latenz je Q-Code, Fehlerquote, letzte Antwort
POST	`/api/sensor`	unverändert, plus Feld `device_id`

Kanonischer Snapshot:

```
{
  "device_id": "st10",
  "ts": "2026-10-17T15:02:11.250Z",
  "age_ms": 820,
  "availability": "AVAILABLE",
```

```
    "execution": "ACTIVE",
    "mode": "AUTOMATIC",
    "program": "O01234",
    "alarm": false,
    "axes": { "X": { "pos_mm": 42.180, "diameter": true }, "Z": { "pos_mm": -12.405
  } },
    "spindle": { "rpm": 1850, "vc_m_min": 245.2 },
    "tool": { "number": 3 },
    "parts": { "count": 117, "last_cycle_s": 94, "prev_cycle_s": 96 },
    "current": { "spindle_a": 11.8, "power_w": null, "load_pct": null }
  }
```

`vc_m_min` ist die einzige echte Ableitung, die bei niedriger Rate trotzdem stimmt: $vc = \pi \cdot D \cdot n / 1000$, mit `D` aus der X-Position (Durchmesser) und `n` aus der Drehzahl. Das ist für die Lehre interessant, weil man G96 (konstante Schnittgeschwindigkeit) direkt sieht. `power_w` und `load_pct` bleiben `null`, bis Spannung, $\cos \phi$ und Nennstrom im Profil stehen; das Frontend zeigt dann „nicht kalibriert" statt einer erfundenen Zahl.

Speicher: Für das MVP bleibt SQLite, hinter einer kleinen `Storage`-Schnittstelle (`write_sample`, `query_series`, `query_states`). Die im Projekt bereits favorisierte InfluxDB 3 Core kommt in Phase 5 als zweite Implementierung dazu. Wer jetzt sofort auf Influx geht, baut zwei Unbekannte gleichzeitig ein; der Adapter ist schon riskant genug.

Zustandsintervalle werden beim Schreiben gebildet, nicht beim Lesen: Ändert sich `execution`, wird das laufende Intervall geschlossen und ein neues geöffnet (`device_states(device_id, state, t_start, t_end)`). Die Zeitleiste und die Verfügbarkeitskennzahl sind dann einfache Abfragen.

---

4. Frontend

4.1 Technologiewahl

Vanilla-JavaScript als ES-Module, kein Build-Schritt, vom Backend ausgeliefert. Dazu **uPlot** für Zeitreihen (klein, schnell bei vielen Punkten) und eine eigene SVG-Komponente für die XZ-Bahn. i18n in ES/EN/DE wie im Referenzansatz vorgesehen.

Gegenposition: React + Vite + TypeScript gäbe Komponentenstruktur, Typprüfung und ein großes Ökosystem. Für ein langlebiges Produktteam wäre das die richtige Wahl. Hier wird das Dashboard aber von wechselnden Studierenden gepflegt und läuft auf einem Raspberry Pi; jede Toolchain, die erst installiert und gepflegt werden muss, ist ein Ausfallrisiko im nächsten Semester. Strukturiert wird stattdessen über klare Modulgrenzen und Web Components für wiederkehrende Bausteine (`<kpi-tile>`, `<state-badge>`, `<signal-chart>`). Typsicherheit kommt über JSDoc-Typen plus `// @ts-check`, was VS Code ohne Build prüft.

Weil das Backend das Frontend selbst ausliefert, entfällt das CORS-Problem, für das `cors_proxy.py` geschrieben wurde.

4.2 Gestaltungsprinzipien

Leitlinie ist das High-Performance-HMI-Konzept nach ISA-101: Grautöne für den Normalzustand, Farbe nur für Abweichung. Ein Dashboard, das im Normalbetrieb bunt ist, kann eine Störung nicht mehr hervorheben.

Zustand	Darstellung
ACTIVE	neutrales Dunkelgrau, ausgefülltes Symbol
READY / IDLE	helles Grau, Umriss
FEED HOLD / STOP	Gelb (Hinweis)
ALARM	Rot, Banner über volle Breite
UNAVAILABLE / Daten veraltet	schraffiert, Wert ausgegraut, Alter sichtbar

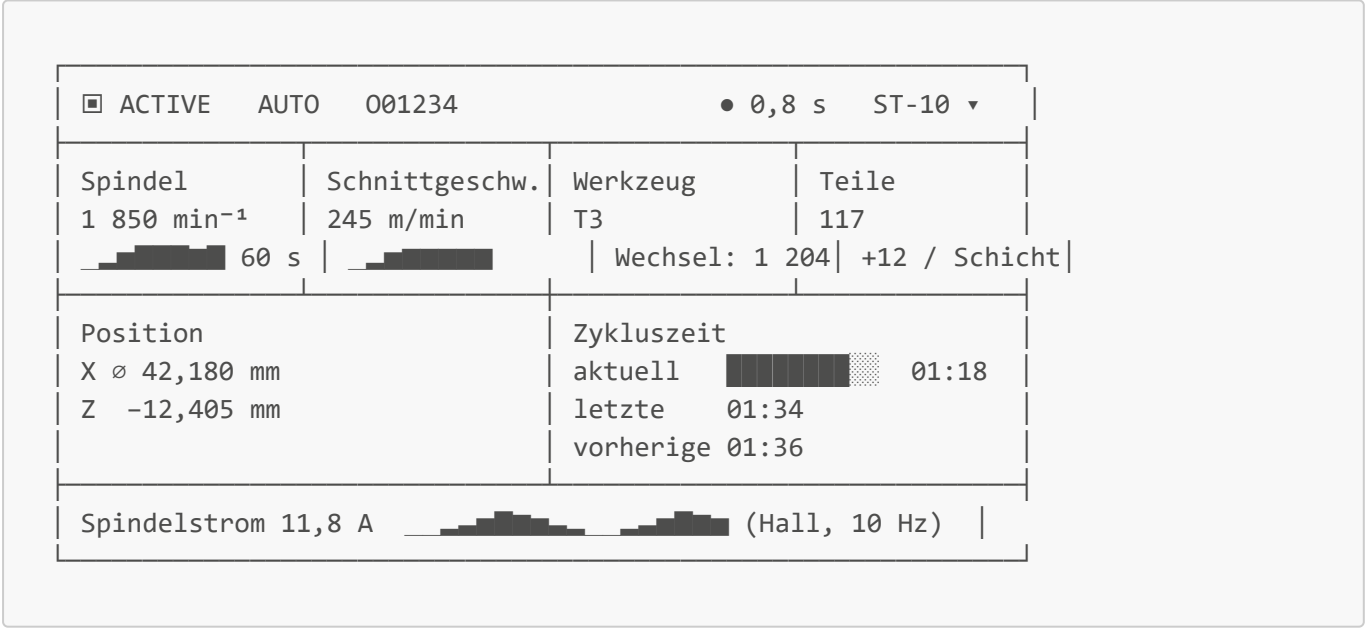
Weitere Regeln: Jede Zahl trägt ihre Einheit. Jede Kachel zeigt, wie alt ihr Wert ist, sobald er älter als das Dreifache der erwarteten Rate ist — bei 2 Hz und serieller Strecke ist „veraltet" ein realer Betriebszustand, kein Randfall. Tabellarische Ziffern (`font-variant-numeric: tabular-nums`), damit springende Werte nicht das Layout wackeln lassen. Dunkel- und Hellmodus über CSS-Custom-Properties; dunkel als Standard an der Maschine, hell für Beamer im Unterricht.

4.3 Ansichten

Fünf Ansichten, oben als Tabs, auf Tablets als untere Navigationsleiste.

Kopfzeile (immer sichtbar): Maschinenname und -bild, Zustands-Badge, Modus, aktives Programm, Verbindungspunkt mit Datenalter, Gerätewahl (ST-10 / TMP1).

1 · Live — die Ansicht für den Bediener und den Rundgang mit Besuchern.



Der Fortschrittsbalken der aktuellen Zykluszeit nutzt die letzte Zykluszeit als Referenz. Überschreitet der aktuelle Zyklus sie um mehr als 10 %, wird der Balken gelb — ein einfacher, aber für Bediener sofort verständlicher Hinweis.

2 · Bahn — XZ-Ansicht in Drehmaschinen-Konvention: Z horizontal (Spindel links), X vertikal. Umschalter Durchmesser/Radius. Die letzten N Punkte als Linie, eingefärbt nach Werkzeugnummer, aktuelle Position als Marker. Hinweis im Panel: „Abtastung ≈ 2 Hz — Ablaufdarstellung, keine Kontur". Ohne diesen Satz interpretiert jemand die eckige Linie als Maßabweichung.

3 · Signale — synchronisierte uPlot-Diagramme untereinander mit gemeinsamer Zeitachse: Drehzahl, X, Z, Spindelstrom, darunter ein Zustandsband. Zeitfenster 5 min / 1 h / Schicht / frei. Zoom in einem Diagramm zoomt alle. Export des sichtbaren Ausschnitts als CSV. Diese Ansicht ist die eigentliche Brücke zur Forschung: Hier lassen sich Stromverläufe pro Werkzeug und Programm vergleichen.

4 · Zustand & Kennzahlen — horizontale Zeitleiste der Zustandsintervalle über die Schicht, darunter Kacheln für Verfügbarkeit (Laufzeit / geplante Zeit), Teile pro Stunde, mittlere Zykluszeit mit Streuung. Hier ehrlich bleiben: Echte OEE braucht Sollzykluszeit und Gutteile. Beides gehört in Odoo. Bis zur Kopplung zeigt die Kachel „Verfügbarkeit", nicht „OEE".

5 · Diagnose — für Entwickler und Studierende: Adapter-Status, Latenz je Q-Code (Median, p95), Timeouts, Fehlerquote, letzte Rohantwort, Agent-Sequenznummern, Zeilenzahlen in der Datenbank, Profil-Datei. Die Ansicht ist absichtlich nüchtern; sie beantwortet „warum steht da nichts?".

4.4 Modulstruktur

```
frontend/
  index.html
  css/
    tokens.css          # Farben, Abstände, Typografie (hell/dunkel)
    layout.css
    components.css
  js/
    main.js             # Router (Hash-basiert), Gerätewahl
    api.js              # REST + SSE, Reconnect mit Backoff, Datenalter
    store.js            # Zustand pro Gerät, Abonnement-Mechanismus
    i18n.js             # ES / EN / DE
    format.js           # Einheiten, Zahlen, Dauer
    views/
      live.js  path.js  signals.js  states.js  diagnostics.js
    components/
      kpi-tile.js  state-badge.js  signal-chart.js  xz-path.js  state-timeline.js
  vendor/
    uPlot.iife.min.js   # lokal, kein CDN – die Zelle hat nicht immer Internet
```

Der `store` ist die einzige Stelle, die Daten hält. Ansichten abonnieren nur, was sie brauchen. Beim Tab-Wechsel wird abbestellt, damit die Pi-Last nicht mit der Zahl geöffneter Ansichten wächst.

Verbindungsverlust: SSE schließt sich → `api.js` versucht es mit 1, 2, 4, … max. 30 s erneut; währenddessen zeigt die Kopfzeile „getrennt seit …", Werte bleiben stehen, aber ausgegraut. Nach dem Reconnect lädt die Signalansicht die Lücke über `/series` nach.

---

5. Implementierungsphasen

Aufwand geschätzt für eine Person mit halber Kapazität. Jede Phase endet mit einem prüfbaren Kriterium; ohne erfülltes Kriterium wird nicht weitergebaut.

Phase	Inhalt	Dauer	Abnahmekriterium
0 · Machbarkeit	Steuerungsgeneration und Softwarestand bestimmen; Setting 143 vorhanden?; RS-232-Belegung (DNC?) klären; Kabel + Moxa anschließen; `Q100`, `Q500`, `Q600 5021` von Hand senden; Latenz je Code messen; Makrovariablen-Liste aus dem Handbuch verifizieren	1 Woche	Protokoll mit gemessener Antwortzeit je Q-Code und bestätigter Variablenliste
1 · Adapter + Agent	`haas_mdc`-Adapter mit Gruppen-Scheduler; SHDR-Server; `Devices.xml`; `cppagent` im Container; 30-min-Mitschnitt an der Maschine; `replay.py`	2 Wochen	`curl :5000/current` liefert X, Z, Drehzahl, Status der ST-10; Replay erzeugt identische Ausgabe
2 · Backend	Namespace-Fix, Device-Profile, `/sample?from=`, kanonischer Snapshot, SSE, Zustandsintervalle, `Storage`-Schnittstelle (SQLite), korrigierte Leistungsformel; Unit-Tests für Parser und Profile	1–2 Wochen	TMP1 läuft unverändert weiter; ST-10 über dieselbe API abrufbar; Tests grün
3 · Frontend-MVP	Kopfzeile, Live-Ansicht, Diagnose-Ansicht, `api.js` mit Reconnect, i18n, Hell/Dunkel	2 Wochen	Live-Ansicht an Maschine und mit Replay; Trennung/Wiederverbindung sichtbar korrekt
4 · Analyse-Ansichten + Hall-Sensor	Bahn, Signale, Zustand & Kennzahlen; Hall-Sensor an `/api/sensor` mit `device_id`; Kalibrierwerte ins Profil	2 Wochen	Synchronisierter Zoom; Strom und Drehzahl zeitlich korrekt übereinander (Abgleich über bekannten Spindelstart)
5 · Härtung	`docker-compose.yml` für Pi (Adapter host-nativ oder privilegiert, Agent/Backend/Influx im Container); InfluxDB-3-Implementierung; Logging-Rotation; README + Studierendenanleitung	1 Woche	Kaltstart der Pi → Dashboard läuft ohne manuellen Eingriff; 24-h-Dauerlauf ohne Speicherwachstum

Gesamt: etwa 9–10 Wochen, also ein Semesterprojekt.

Zur Docker-Frage passt die bereits im Projekt notierte Richtung: Signalaufnahme host-nativ, Verarbeitung im Container. Für den Adapter heißt das konkret: Mit Moxa (TCP) kann er problemlos in einen Container; bei direktem USB-Seriell-Adapter am Pi läuft er besser als `systemd`-Dienst.

---

6. Repository-Struktur

Statt eines zweiten Ordners `IIoT_Haas_ST10` neben `IIoT_Haas` wird `dashboard/` so umgebaut, dass beide Maschinen denselben Code nutzen:

```
dashboard/
  adapters/
    haas_mdc/                # neu, ST-10
  agent/
    agent.cfg
    devices/st10.xml
  backend/                   # aus IIoT_Haas/Backend/main.py hervorgegangen
    app.py  poller.py  profiles.py  storage/  api/
    tests/
  profiles/
    haas_ngc_tmp1.yaml
    haas_chc_st10.yaml
  frontend/
  tools/
    cors_proxy.py            # nur noch Dev-Fallback
  docker-compose.yml
  README.md
```

`IIoT_Haas/` bleibt bis zum Ende von Phase 2 stehen und wird dann in `backend/` überführt, damit der laufende TMP1-Betrieb nicht unterbrochen wird.

---

7. Teststrategie

Parser und Profile werden mit eingefrorenen XML-Antworten getestet (eine NGC-Antwort der TMP1, eine `cppagent`-Antwort der ST-10). Der Adapter wird gegen `replay.py` getestet, inklusive absichtlich kaputter Antworten (abgeschnittene Zeile, fehlendes ETB, Timeout). Für das Frontend genügt in diesem Umfang ein manueller Prüfplan pro Ansicht plus ein Playwright-Smoke-Test, der prüft, dass jede Ansicht mit Replay-Daten ohne Konsolenfehler rendert.

---

8. Offene Punkte

1. Ist die ST-10 tatsächlich eine CHC vor 2016? Falls NGC: Phasen 0–1 entfallen, Port 8082 (MTConnect) bzw. 5051 (Q-Codes über Ethernet) prüfen.
2. Ist der RS-232-Port für DNC belegt?
3. Welche Makrovariable liefert die Spindellast an dieser Steuerung, falls überhaupt?
4. Typenschilddaten des Spindelmotors (Nennstrom, `cos ϕ`) und tatsächliche Netzspannung am Schaltschrank.
5. Wo kommt die Sollzykluszeit für eine echte OEE her — direkt aus Odoo-Arbeitsaufträgen oder vorerst manuell im Profil?
6. Soll das Dashboard später auch Daten an das Simulationsmodell (Process Simulate / Simio) liefern? Falls ja, ist der MTConnect-Agent der natürliche Abgriffspunkt, nicht die Backend-Datenbank.

Quellen

- Referenz-Backend: ThMRudolf/2023_ProdcutionCellITAM — dashboard
- Serielle MDC-Parameter (38400 Bd, 7E1, XON/XOFF, Setting 143), Vor-2017-Maschinen nur seriell, Verbindungslimit: MachineMetrics — Haas Serial Machine Data Collection
- TrakHound-Adapter (MIT, Ethernet-Modul Port 5051): TrakHound/MTConnect.NET-Haas
- Haas MDC für NGC: Haas — Machine Data Collection NGC